

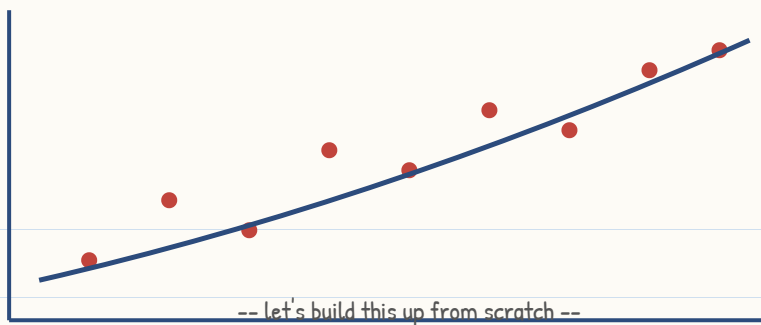
Linear Regression – A to Z

complete notes: beginner → advanced, interview-ready, data scientist lens

what's inside ↓

1. what is linear regression + types pg 2
2. the math – hypothesis, cost, MLE link pg 3
3. optimization – normal eq, GD variants, learning rate pg 4
4. assumptions + diagnostics (leverage, Cook's D) pg 5
5. statistical inference – t-test, F-test, AIC/BIC pg 6
6. feature engineering – scaling, encoding, polynomial pg 7
7. regularization – ridge / lasso / elasticnet + tuning pg 8
8. outliers & robust regression pg 9
9. full end-to-end code pipeline pg 10
10. evaluation metrics + bias-variance pg 11
11. when to use LR vs other models + prod gotchas pg 12
12. interview Q&A – beginner → advanced pg 13
13. one-page cheat sheet pg 14

goal: not just pass the interview – actually *understand* it well enough to debug it when it breaks in prod.



beginner

1. what is linear regression?

a **supervised learning** algorithm that models the relationship between input features (X) and a **continuous target (y)** by fitting a straight line (or hyperplane) through the data.

used for: predicting *numbers* — house prices, sales forecasts, salary estimates, demand, temperature, construction cost per sqft, etc. **NOT** for classification (that's logistic regression).

the 3 flavors

type	formula shape	example
simple linear	1 feature: $\hat{y} = \beta_0 + \beta_1 x$	predict price from sqft alone
multiple linear	many features: $\hat{y} = \beta_0 + \sum \beta_i x_i$	price from sqft + rooms + location
polynomial	$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots$	curved relationships (still "linear" in the coefficients!)

→ interview trap: "is polynomial regression still linear regression?" YES — it's linear in the *parameters* β , even though the curve isn't straight. that's the actual definition of "linear" here.

why data scientists still reach for it (2026, post-deep-learning era)

- **interpretability** — you can explain exactly why a prediction was made (regulators/stakeholders love this)
- **baseline model** — always fit one first; if a fancy model can't beat it, something's wrong
- **fast & cheap** — trains in milliseconds, easy to retrain frequently in prod
- **statistical rigor** — gives you p -values, confidence intervals — actual uncertainty estimates, which most ML models don't

the assumptions in one line (detailed on pg 5)

L.I.N.E. — *Linearity, Independence, Normality of residuals, Equal variance (homoscedasticity)*, + no multicollinearity.

mnemonic: "LINE" — keeps you from blanking in interviews

2. the math underneath

hypothesis function

$$\hat{y} = X\beta \quad (\text{matrix form, } X \text{ includes a column of 1's for } \beta_0)$$

cost function — Mean Squared Error

$$J(\beta) = (1/2m) \sum (\hat{y}_i - y_i)^2$$

the 1/2 is just a calculus convenience — it cancels the 2 that pops out when you differentiate the square. doesn't change where the minimum is.

the advanced nugget: least squares = Maximum Likelihood Estimation

if you assume the errors $(y - \hat{y})$ are normally distributed with mean 0 and constant variance, maximizing the likelihood of the data is mathematically identical to minimizing the sum of squared errors. this is why squared error was chosen, not just "because it's differentiable" — it's the MLE solution under a Gaussian noise assumption.

top-tier interview Q: "derive why OLS = MLE under gaussian errors."

→ write likelihood of gaussian, take log, log-likelihood has a

$-\sum (y - \hat{y})^2$ term — maximizing that = minimizing SSE. that's the link!

solving for β — the derivative (gradient)

$$\partial J / \partial \beta = (1/m) X^T (X\beta - y)$$

set this to 0 and solve algebraically → gives the **normal equation**. or, don't set to 0 — instead walk downhill using this gradient repeatedly → that's **gradient descent** (next page).

vector calculus intuition (often skipped, often asked)

think of $J(\beta)$ as a bowl-shaped surface in β -space. the gradient $\partial J / \partial \beta$ always points in the direction of steepest increase. so we move in the **opposite** direction to decrease error fastest — that minus sign in the GD update rule isn't arbitrary.

intermediate

3. optimization – solving for β

method	formula / idea	pros	cons
normal equation	$\beta = (X^T X)^{-1} X^T y$	exact, no tuning	$O(n^3)$, needs invertible $X^T X$
batch GD	use ALL data per step	stable convergence	slow on big data
stochastic GD	use 1 random row per step	fast, escapes local dips	noisy path, never fully settles
mini-batch GD	use small batches (32-256)	best of both – used in practice	needs batch-size tuning

learning rate (α) – the single most-tuned hyperparameter



→ in practice: try $\alpha = 0.001, 0.01, 0.1, 1$ (log scale), watch the loss curve. if loss increases or oscillates → α too high.

convergence – how do you know GD is "done"?

- cost function stops decreasing meaningfully between iterations (below a threshold ϵ)
- or you hit a max iteration cap
- plot cost vs iteration — should always be monotonically decreasing for batch GD (if it's not, α is too big)

feature scaling speeds up GD a LOT — unscaled features create an elongated bowl → zig-zag path.
scaled features = round bowl → direct path to min.

4. assumptions + diagnostics

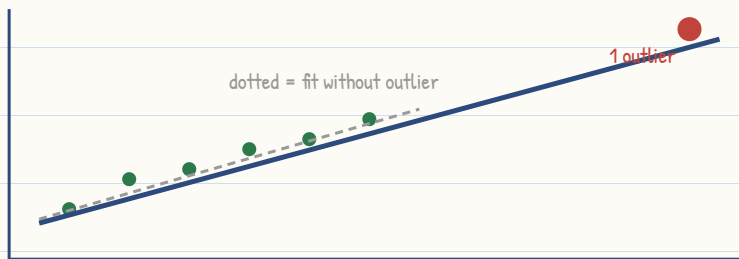
- ✓ **linearity** — X - y relationship is linear → check: scatter / residual-vs-fitted plot
- ✓ **independence** — residuals uncorrelated → check: Durbin-Watson test (breaks w/ time series)
- ✓ **homoscedasticity** — constant residual variance → check: residual plot shouldn't funnel
- ✓ **normality of residuals** — needed for valid CIs/p-values, not for prediction → check: Q-Q plot
- ✓ **no multicollinearity** — features not redundant → check: VIF > 5-10 is a red flag

influence diagnostics (the part beginners skip, seniors ask about)

concept	what it measures	rule of thumb
leverage (h_{ii})	how extreme a point's X values are	high leverage ≠ bad, just unusual X
studentized residual	how far off the prediction is, standardized	$ value > 3$ = possible outlier
Cook's distance	how much removing this point would change the whole model	$> 4/n$ = influential, investigate it

a point can have high leverage but low influence (it agrees with the trend), or low leverage but still shift the model a lot if it's a residual outlier. **Cook's distance combines both** — that's why it's the go-to metric.

visual: leverage point pulling the line



advanced

5. statistical inference (what makes LR different from other ML)

unlike most ML models, linear regression gives you formal **hypothesis tests** on whether each feature actually matters.

t-test on each coefficient

$H_0: \beta_i = 0$ (feature has no effect) vs $H_1: \beta_i \neq 0$

test statistic: $t = \beta_i / SE(\beta_i) \rightarrow$ get a p-value from the t-distribution

$\rightarrow p < 0.05$ usually means "keep this feature, it's statistically significant." but always sanity-check against domain knowledge too.

F-test — is the model as a whole useful?

$H_0: \text{all } \beta_i = 0$ (model explains nothing) vs $H_1: \text{at least one } \beta_i \neq 0$. a significant F-stat means your model beats a "just predict the mean" baseline.

R^2 vs adjusted R^2 vs AIC/BIC

metric	use for
R^2	quick "how much variance explained" — but always $\uparrow$ with more features
adjusted R^2	penalizes useless features — better for comparing models w/ different # features
AIC / BIC	model selection balancing fit vs complexity; lower is better. BIC penalizes complexity harder than AIC

confidence interval on a coefficient

$$\beta_i \pm t^* \times SE(\beta_i)$$

e.g. "each extra sqft adds ₹1,200 to price, 95% CI [₹950, ₹1,450]" — this is the kind of statement that makes stakeholders trust the model.

senior-level interview Q: "your model has $R^2=0.9$ but a feature's p-value is 0.4 — what do you do?" $\rightarrow$ consider dropping it (weak evidence it truly matters), check VIF too (multicollinearity inflates p-values).

6. feature engineering

scaling — standardize vs normalize

method	formula	when
standardization (z-score)	$(x - \mu) / \sigma$	default choice; needed for Ridge/Lasso, GD-based fitting
min-max normalization	$(x - \min) / (\max - \min)$	when you need bounded [0,1] range

plain OLS via normal equation is technically scale-invariant in its predictions, but you should **still scale** if comparing coefficient magnitudes, or if going anywhere near regularization / gradient descent.

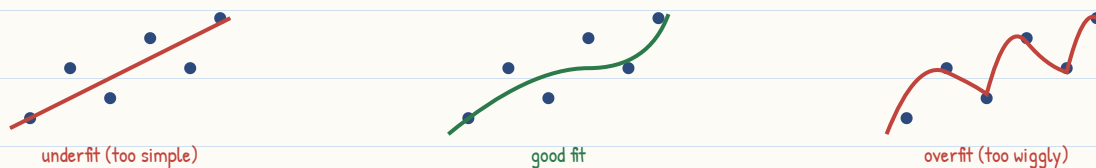
encoding categorical variables

method	use when	watch out
one-hot encoding	nominal categories (city, color)	drop 1 category — avoid dummy variable trap
label / ordinal encoding	categories have real order (low/med/high)	don't use on nominal data — implies false order
target encoding	high-cardinality categoricals	leaks target info — fit only on train folds!

polynomial & interaction features

add x^2 , x^3 , or $x_1 x_2$ terms to capture curvature / combined effects — still fit with plain linear regression since it's linear in the coefficients.

underfitting vs good fit vs overfitting



→ high-degree polynomial features overfit fast — always validate on a held-out set before adding more.

intermediate → advanced

7. regularization

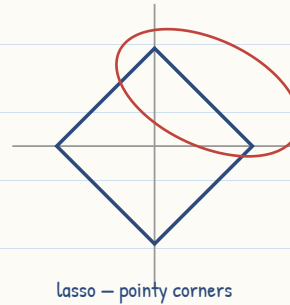
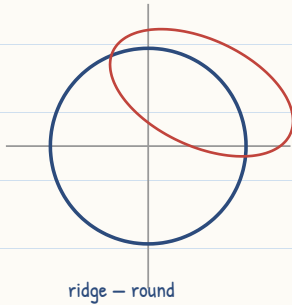
too many features / multicollinearity → OLS overfits. add a penalty term:

ridge (L2): $J = SSE + \lambda \sum \beta_i^2$ → shrinks coefs toward 0, never exactly 0

lasso (L1): $J = SSE + \lambda \sum |\beta_i|$ → can zero coefs out = auto feature selection

elasticnet: $\lambda_1 \sum |\beta| + \lambda_2 \sum \beta^2$ — mix of both, good when features are correlated AND you want sparsity

geometric intuition



the error ellipse (red) first touches lasso's diamond at a **corner** → a coordinate = 0 → feature dropped. ridge's circle has no corners, so coefs shrink but rarely hit exactly 0.

hyperparameter tuning λ (called alpha in sklearn)

```
from sklearn.linear_model import RidgeCV, LassoCV
# CV automatically tests a range of alpha (lambda) values
ridge = RidgeCV(alphas=[0.01, 0.1, 1, 10, 100], cv=5).fit(X_train_s, y_train)
print(ridge.alpha_) # best lambda found
```

$\lambda=0$ → plain OLS. $\lambda \rightarrow \infty$ → all coefs shrink to
~0 (underfits). sweet spot found via cross-validation.

advanced

8. outliers & robust regression

why OLS is fragile: squared error means one extreme point contributes a HUGE amount to total loss → the line bends toward it.

detecting outliers/influential points

- studentized residuals $> |3|$ → flag as potential outlier
- Cook's distance $> 4/n$ → point is meaningfully swaying the whole fit
- simple: boxplot / z-score > 3 on residuals

what to actually do about them

option	when
investigate & fix (data entry error?)	always do this first — don't blindly delete
remove the point	confirmed error / not representative of the real population
transform target ($\log(y)$)	right-skewed target w/ a few huge values (e.g. income, house price)
use robust regression	outliers are real but you don't want them dominating

robust regression — Huber loss

Huber loss = squared error for small residuals, linear (absolute) error for large residuals — best of both: smooth near zero, doesn't explode for outliers.

```
from sklearn.linear_model import HuberRegressor
model = HuberRegressor(epsilon=1.35).fit(X_train_s, y_train)
# epsilon controls the switch point between squared & linear loss
```

RANSAC — another robust option

fits the model repeatedly on random subsets, keeps the fit that has the most "inlier" agreement. good when a large fraction of your data might be corrupted/noisy (e.g. sensor glitches).

interview framing: "I'd first ask *why* the outlier exists — data bug vs real rare event — before deciding to remove, transform, or switch to Huber/RANSAC." shows judgment, not just a recipe.

hands-on

9. full end-to-end pipeline

```
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score

# 1. split first, always
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. preprocessing per column type -- avoids leakage, keeps code clean
preprocess = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), cat_cols),
])

# 3. bundle preprocessing + model into ONE pipeline object
pipe = Pipeline([
    ("prep", preprocess),
    ("model", Ridge()),
])

# 4. tune alpha via cross-validation -- grid search over the WHOLE pipeline
param_grid = {"model__alpha": [0.01, 0.1, 1, 10, 100]}
search = GridSearchCV(pipe, param_grid, cv=5, scoring="neg_root_mean_squared_error")
search.fit(X_train, y_train)

print("best alpha:", search.best_params_)

# 5. evaluate on untouched test set
y_pred = search.predict(X_test)
print("test RMSE:", mean_squared_error(y_test, y_pred, squared=False))
print("test R2:", r2_score(y_test, y_pred))
```

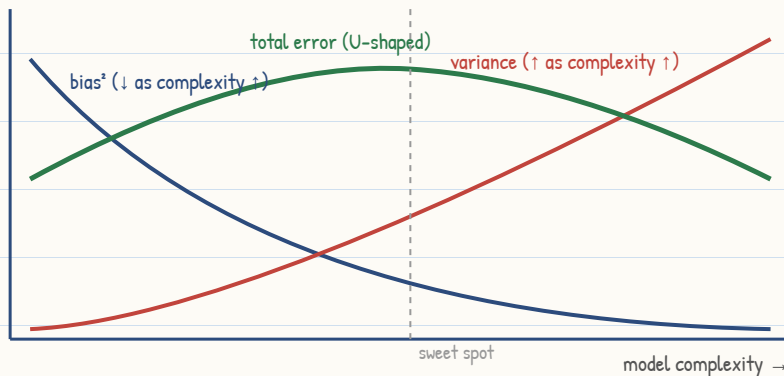
using Pipeline + ColumnTransformer isn't just "clean code" –
it PREVENTS leakage automatically: CV folds re-fit the scaler
& encoder fresh on each fold's training data only. huge in interviews.

intermediate

10. evaluation metrics + bias-variance

metric	tells you	watch out
MAE	avg abs error, same units as y	doesn't punish big misses much
MSE / RMSE	punishes large errors more	sensitive to outliers
R^2	% variance explained	always $\uparrow$ when adding features
adj. R^2	penalized for useless features	use to compare models fairly
MAPE	% error — business-friendly	breaks down near $y=0$

the bias-variance tradeoff — the deepest idea in ML



plain OLS with too many features = low bias, high variance (overfits — great on train, bad on test). regularization deliberately adds a little bias to buy a big reduction in variance — usually a net win on unseen data.

the fix: cross-validation, not just a single train/test split

k -fold CV (typically $k=5$ or 10) gives a much more reliable estimate of test performance than one split — reduces the chance you got lucky/unlucky with a particular split.

11. when to use LR vs other models + prod reality

situation	go with
need interpretability / regulated domain (finance, healthcare)	linear regression
relationship is genuinely non-linear, no time to engineer features	tree-based (RF, XGBoost)
huge unstructured data (images, text)	deep learning
small dataset, need to ship fast, need a baseline	linear regression
lots of categorical interactions, no assumptions wanted	gradient boosting

→ always fit linear regression FIRST as a baseline, even if you know you'll end up using XGBoost. if XGBoost only beats it by 1% RMSE, the added complexity may not be worth it.

production gotchas (what separates "did a course" from "shipped it")

- **model drift** — feature distributions shift over months; monitor prediction error over time, not just at launch
- **correlation ≠ causation** — coefficient = "associated with," not "causes." critical when giving business recommendations
- **categorical drift** — new category appears at inference time that wasn't in training → use `handle_unknown="ignore"` in `OneHotEncoder`
- **retraining cadence** — decide upfront: daily/weekly/monthly retrain based on how fast the domain changes
- **feature availability at inference** — a feature that's easy to get in training (e.g. computed post-hoc) might not exist in real-time serving. check this BEFORE building the model
- **extrapolation risk** — LR predictions get unreliable outside the range of training data. always log when inputs are out-of-distribution

"feature availability at inference" is the #1 thing juniors miss and the #1 thing that breaks real pipelines. always ask:
"will I actually have this value at prediction time, in real-time?"

12. interview Q&A – by tier

beginner

Q: what's the difference between correlation and regression?

A: correlation measures strength/direction of a relationship (symmetric). regression models one variable to predict another (asymmetric, directional).

Q: why minimize squared error, not absolute error?

A: squared error is differentiable everywhere (needed for gradient-based optimization) and penalizes large errors more heavily.

intermediate

Q: how do you detect & fix multicollinearity?

A: $VIF > 5-10$ flags it. fix: drop/combine correlated features, PCA, or use Ridge regression.

Q: $R^2 = 0.95$ — good model?

A: not necessarily — check test-set R^2 (not train), and RMSE in real units. high R^2 can mean overfitting or an easy target.

Q: when would you pick Lasso over Ridge?

A: when you suspect many features are irrelevant and want automatic feature selection / a sparse, interpretable model.

advanced

Q: derive why OLS = MLE under Gaussian noise.

A: write the Gaussian likelihood of residuals, take the log — the log-likelihood contains a $-\sum(y-\hat{y})^2$ term. maximizing likelihood = minimizing SSE.

Q: your coefficient has a low p-value but the confidence interval is huge — what's going on?

A: possible multicollinearity inflating standard errors, or small sample size. check VIF and sample size before trusting the point estimate.

Q: how does Cook's distance differ from just looking at residuals?

A: residuals only capture how "wrong" a prediction is; Cook's distance also factors in leverage (how extreme the X values are) — combining both tells you how much removing that point would actually change the fitted model.

Q: bias-variance tradeoff, precisely?

A: total expected test error = $\text{bias}^2 + \text{variance} + \text{irreducible noise}$. simple models (few features / high λ) → high bias, low variance. complex models → low bias, high variance. regularization trades a bit of bias for a lot less variance.

13. one-page cheat sheet

model: $\hat{y} = \beta_0 + \sum \beta_i x_i$ | **cost:** $J = (1/2m) \sum (\hat{y} - y)^2$ | **solve:** normal eq (small data) or gradient descent (large data)

assumptions — "LINE"

Linearity · Independence of errors · Normality of residuals · Equal variance (homoscedasticity) · + no multicollinearity

checks

linearity/homoscedasticity → residual plot | normality → Q-Q plot | multicollinearity → VIF (>5-10 = bad) | influence → Cook's distance (>4/n = flag)

regularization

Ridge (L2) → shrinks, never 0 | Lasso (L1) → can zero out, feature selection | ElasticNet → both | tune λ via CV

metrics

MAE (robust) · RMSE (punishes big errors) · R^2 (variance explained, always ↑) · adj. R^2 (fair comparison) · AIC/BIC (lower=better, model selection)

pipeline checklist

- ☒ train/test split BEFORE any fitting
- ☒ scale: fit on train only, transform test
- ☒ encode categoricals (one-hot, drop first)
- ☒ check assumptions on residuals
- ☒ cross-validate to tune hyperparameters
- ☒ evaluate on untouched test set
- ☒ monitor for drift once in production

golden interview lines to remember

" β_i = effect on y per unit x_i , holding others constant."
"coefficient = association, not causation."
"OLS under gaussian-error assumption IS maximum likelihood."
"always fit linear regression first as a baseline."